Technical Report — SyncSpace (Project 3)

Course: S26CS6.401 — Software Engineering
Project: SyncSpace — real-time collaborative code editor (CRDT + WebSocket relay)
Repository: SE 3

This report consolidates **design decisions**, **architecture**, **prototype implementation**, **quantitative analysis** against non-functional requirements, comparison to an **alternative architectural pattern**, **process reflections**, and **individual contributions**.

---

1. Executive summary

SyncSpace lets several users edit one shared source document in the browser with **low perceived latency**. Each browser holds a **Yjs** CRDT replica; edits apply locally first, then propagate as **binary update messages**. A **FastAPI** server does **not** merge or validate edits—it only **groups WebSockets by session** and **broadcasts** frames. **Awareness** (cursors, names) uses the same socket with a **one-byte prefix** so document and presence streams stay distinguishable. The prototype intentionally implements **one end-to-end capability**—real-time co-editing with presence—rather than the full SRS (e.g. multi-language highlighting beyond JavaScript, production TLS).

---

2. Requirements and architectural drivers

Detailed functional and non-functional requirements, subsystem decomposition, and rationale for architecturally significant requirements are in `Task 1/SRS.md` and `Task 1/system.md`.

Drivers retained in the implementation:

- **Local-first editing** (FR-CE-01) → Yjs on the client; no server round-trip before rendering a keystroke.
- **Deterministic conflict handling** (FR-CE-03) → CRDT instead of ad-hoc merging.
- **Live collaboration** (FR-CE-02, FR-PA-01) → WebSocket relay + Yjs Awareness + CodeMirror `y-codemirror.next`.
- **Simple operations** (deployment concern) → stateless relay; session routing only.

---

3. Architecture framework

3.1 Stakeholders (IEEE 42010)

Stakeholders, concerns, viewpoints, and views are documented in `Task 2/Stakeholders.md`.

3.2 Major design decisions (ADRs)

Eight **Nygard-style** ADRs are in `Task 2/ADR/` (see `ADR.md` for the index). The following are representative:

ADR	Decision
ADR-001	Yjs (CRDT) over operational transformation
ADR-002	WebSocket over HTTP polling

ADR-003	Web editor over VS Code extension
ADR-004	Stateless relay over centralized document authority
ADR-005	Python and FastAPI for the relay service
ADR-006	Capability URLs for sessions (no login) in the prototype
ADR-007	No database for the MVP relay (ephemeral sessions)
ADR-008	Versioned relay/client WebSocket framing and implementation details

3.3 Tactics and patterns

- **Tactics (4–5):** **`Task 3/Arch tactics.md`** — aligned with the **as-built** prototype (relay forwards only; catch-up via **peers**, not server-side Y.Doc).
- **Patterns (2):** **`Task 3/Impl patterns.md`** — Observer (Yjs ↔ editor), Mediator (relay).

---

4. Implementation

4.1 Structure

Component	Location	Role
Relay + static host	`Implementation/src/server/server.py`	HTTP redirect to `/app/?session=…`, `StaticFiles` for client, `/ws/{session_id}` broadcast
Client	`Implementation/src/client/`	CodeMirror, Yjs, custom `SimpleProvider` for framing
Dependencies	`Implementation/requirements.txt`	FastAPI, Uvicorn, websockets (tests)

4.2 Session and transport

- Visiting `/` issues a **307** redirect to `/app/?session=<id>` (server-generated UUIDv4).
- Clients open `ws://<host>/ws/<session_id>` (or `wss` when deployed with TLS).
- **Binary protocol:** `0x00` + Yjs update, `0x01` + awareness update; **JSON** text is used only for `request_state` so new joiners can obtain a **full state** from existing peers (the server does not store document bytes).

4.3 Divergence from an “ideal” full system

The SRS mentions heartbeat timeouts, exponential backoff, and optional server-side state for diffs. The **prototype** uses:

- **Browser WebSocket** close events; on disconnect the client **reloads after 5 s** (simple recovery, not full backoff spec).
- **Peer-to-peer state exchange** for new members instead of a server-held Y.Doc.

These choices reduce server complexity and match **ADR-004**; they are called out so grading can distinguish **documentation vision** from **delivered prototype**. The full set of consciously accepted trade-offs (ephemeral sessions, reload-based recovery, trusted-client assumption, semantic conflict blindspot) and the explicit trust model are documented in **`Task 1/SRS.md`** **§§ 3.7–3.8**.

---

5. Architecture analysis: comparison to an alternative pattern

5.1 Chosen pattern (as implemented): CRDT + stateless relay

- **Correctness:** Convergence is delegated to **Yjs** on each replica; the relay cannot corrupt merge order because it does not merge.
- **Availability:** No single component must run **operational transformation** or hold authoritative document state.
- **Scalability of relay logic:** O(n) fan-out per message per session; acceptable for the **≤3 users** target.

5.2 Alternative pattern: operational transformation (OT) with central sequencer

A **single authoritative server** (or primary) receives all operations, **transforms** them against concurrent operations in a **total order**, then broadcasts **canonical** operations.

Criterion	CRDT + stateless relay (SyncSpace)	OT + central server
Server complexity	Low (forward only)	High (transform + ordering)
Offline / partition	CRDT designed for eventual merge	OT typically needs reconnect protocol & history
Latency perception	Excellent (local-first)	Depends on RTT to authority for confirmed ops
Consistency story	Eventual convergence (CRDT proof)	Immediate single ordering if server always up

Trade-off summary: We traded a **powerful central coordinator** for **implementation simplicity** and **local-first UX**, which fits a semester prototype and NFR-P-02-style responsiveness. A production Google-Docs-scale system might combine CRDT/OT hybrids or richer server roles; that is out of scope here.

5.3 Second alternative: same CRDT, different integration — Mediator (WebSocket) vs Publish–Subscribe (HTTP)

The primary submission (**Implementation 1**) uses a **WebSocket Mediator** (Task 3): each client holds one **duplex** connection; the server forwards bytes between sockets. `Implementation3/` keeps **Yjs + the same document/awareness payloads** but implements **Publish–Subscribe** instead: clients **POST** an envelope to the server, and receive peer updates on a **Server-Sent Events (SSE)** subscription. The server is a per-session **topic broker** (queue per **connection id**), not a WebSocket endpoint.

Criterion	WebSocket Mediator (Impl 1)	HTTP POST + SSE (Impl 3)
Request style	Long-lived, bidirectional	Request/response (POST) + one-way **push** (SSE)
Typical use	Low-latency, bursty binary streams	Firewalls that restrict WebSockets; simpler HTTP-only tooling — at the cost of **more HTTP overhead** and explicit **per-tab** send ordering in the client

Server shape	`receive` / `send` on sockets	In-memory **fan-out to subscriber queues**

Rationale, trade-offs, and code pointers: **`Implementation3/doc/arch/pattern-swap-impl3.md`** , smoke test **`Implementation3/tests/sse_pubsub_smoke.py`** , default port **8082** (run separately from **8080** / **8081**).

---

6. Quantified non-functional properties (prototype)

Measurements cover **two NFR families** from `Task 1/SRS.md` : **responsiveness / latency** (NFR-P-01, NFR-P-02) and **throughput** as a scalability proxy (NFR-SC-01).

Method: `compare_benchmark.py` (SE-3 root) targets all three running servers simultaneously and collects **40** HTTP-redirect samples, **40** WebSocket/SSE 1-hop latency samples, and **400** sequential throughput updates per implementation. The server machine (`10.2.128.193`) ran all three implementations concurrently; the benchmark client ran on a separate laptop on the same LAN. Numbers vary by hardware, OS, and network path.

Confirmation model per implementation (affects what the latency and throughput numbers measure):

Implementation	Confirmation used	What is timed
Impl-1 CRDT + Relay	Receiver delivery	Sender → relay → **receiver gets frame**
Impl-2 OT + Sequencer	Server ack	Sender → server transforms → **ack back to sender**
Impl-3 Pub-Sub + SSE	Receiver delivery	Sender POST → broker queues → **SSE event at receiver**

Sample run — 2026-04-24, two-laptop LAN (server: `10.2.128.193`):

HTTP redirect latency — 40 samples (lower is better)

Implementation	Mean
Impl-1 CRDT + Stateless Relay	**24.7 ms**
Impl-2 OT + Central Sequencer	35.0 ms
Impl-3 Pub-Sub + SSE	42.7 ms

WebSocket / SSE 1-hop latency — 40 samples (lower is better)

Implementation	Mean	Median	Min	Max	Stdev
Impl-1 CRDT + Relay	70.4 ms	60.7 ms	44.7 ms	117.3 ms	21.4 ms
Impl-2 OT + Sequencer	**27.6 ms**	**8.1 ms**	**5.2 ms**	**97.9 ms**	29.9 ms
Impl-3 Pub-Sub + SSE	123.2 ms	95.4 ms	60.1 ms	242.9 ms	60.0 ms

Throughput — 400 sequential updates (higher is better)

Implementation	ops/s
Impl-2 OT + Central Sequencer	**167.4**
Impl-1 CRDT + Stateless Relay	14.0
Impl-3 Pub-Sub + SSE	11.4

Interpretation:

1. **HTTP latency** ranks CRDT first (24.7 ms) because the relay's `/` handler is a trivial redirect with no state or transformation. OT and Pub-Sub add slightly more startup work per request.
2. **1-hop latency** — Impl-2 OT reports the lowest mean (27.6 ms) because its confirmation is the **server ack**, which returns before the broadcast reaches the second peer. Impl-1 CRDT (70.4 ms) and Impl-3 Pub-Sub (123.2 ms) measure full **receiver delivery**, a longer path. The large stdev and max on OT indicate occasional transform spikes. Pub-Sub's 123 ms mean reflects the HTTP POST round-trip overhead on top of the SSE push — a structural cost of replacing a duplex channel with separate publish and subscribe legs.
3. **Throughput** — OT's ack-gated sequential model (167 ops/s) appears highest because each cycle ends at the server ack, not at receiver delivery; the network leg to the receiver is hidden. CRDT relay (14 ops/s) and Pub-Sub (11 ops/s) both measure end-to-end delivery, so the full LAN RTT is included twice per cycle (send + receive). On loopback, CRDT relay reaches ~7 000 msg/s (see §6 loopback note below), confirming the bottleneck on the LAN run is network RTT, not server CPU.
4. **NFR compliance over the LAN:**
   - NFR-P-01 (E2E sync ≤ 500 ms): all three pass (max observed 243 ms for Pub-Sub).
   - NFR-P-02 (keystroke-to-render ≤ 16 ms): CRDT passes because edits apply locally before any network send; OT and Pub-Sub do not apply the edit locally first, so perceived latency equals the measured 1-hop figure.
   - NFR-SC-01 (≥ 3 concurrent users): all three relay/broker designs handle 3 users within the measured throughput budgets.

Loopback baseline (same machine, ports 8971–8972, from the previous `compare_benchmark.py` run):

Metric	CRDT Relay	OT Sequencer
WS latency median	0.245 ms	1.099 ms
Throughput	7 023 ops/s	5 254 ops/s

Loopback eliminates network RTT and shows pure server overhead: the stateless relay adds ~0.25 ms per hop; the OT sequencer adds ~1.1 ms (JSON parse + transform + apply + ack). Over the LAN those numbers are dominated by the ~20–60 ms RTT, confirming the relay/sequencer is not the bottleneck at prototype scale.

---

7. Reflections and lessons learned

- **Framing matters:** Mixing Yjs updates and awareness on one WebSocket required an explicit **prefix byte**; forgetting it on some code paths broke convergence—good lesson in **protocol discipline** for "simple" relays.
- **Docs vs. code:** Early docs assumed **server-side** state vectors and app-level heartbeats; the shipped prototype is **leaner**. Updating **tactics** and **subsystem** descriptions to match reality avoided misleading evaluators.
- **Scope control:** Delivering **one strong E2E flow** (co-edit + presence) satisfied the prototype bar without implementing every SRS "Should" item.
- **Testing:** Automated **smoke** (`ws_relay_smoke.py`) and **benchmarks** (`benchmark_nfr.py`) made regressions in the relay obvious and gave concrete numbers for the report.

---

8. Individual contributions

Team 33. Only one member uploads to Moodle; this table must still list everyone with a substantive role. Entries below are aligned with **git history**; adjust wording if you split work differently.

Team member	Primary areas	Specific contributions
Vijay A	Core implementation, build/run tooling, documentation integration	FastAPI relay and static hosting, Yjs/WebSocket client (`SimpleProvider` framing), Makefile/`start` scripts, README, stakeholder/ADR doc merges, cross-cutting fixes and project narrative.
Aryan Mishra	Architecture and report	ADR and Technical report updates aligned with the as-built system.
Hardik Chadha	Testing and Architecture Analysis	Implementation 2, Test nfr

---

9. References to repository artifacts

- Requirements: `Task 1/SRS.md` , `Task 1/system.md`
- Stakeholders: `Task 2/Stakeholders.md`
- ADRs: `Task 2/ADR/`
- Tactics / patterns: `Task 3/Arch tactics.md` , `Task 3/Impl patterns.md`
- Code: `Implementation/src/`
- Optional comparison builds: `Implementation2/` (OT + sequencer), `Implementation3/` (pub-sub + SSE; pattern swap in §5.3)
- Setup: **`README.md`**